

Week-6 (DAY-1) Hands-On

Problem: 1- SRP – Single Responsibility Principle

Scenario: Student Report Generator

A training institute like Codempower Academy maintains student information and generates performance reports. Currently, one class performs student data storage and report generation, which makes the code difficult to maintain.

Requirements:

1. Create a **Student class** with properties:
 - StudentId
 - StudentName
 - Marks
2. Create a class responsible for **managing student data**.
3. Create a separate class responsible for **generating reports**.
4. The report should display:

Security Requirements (Secure Coding Practices)

Students must implement the following security measures:

Technical Constraints:

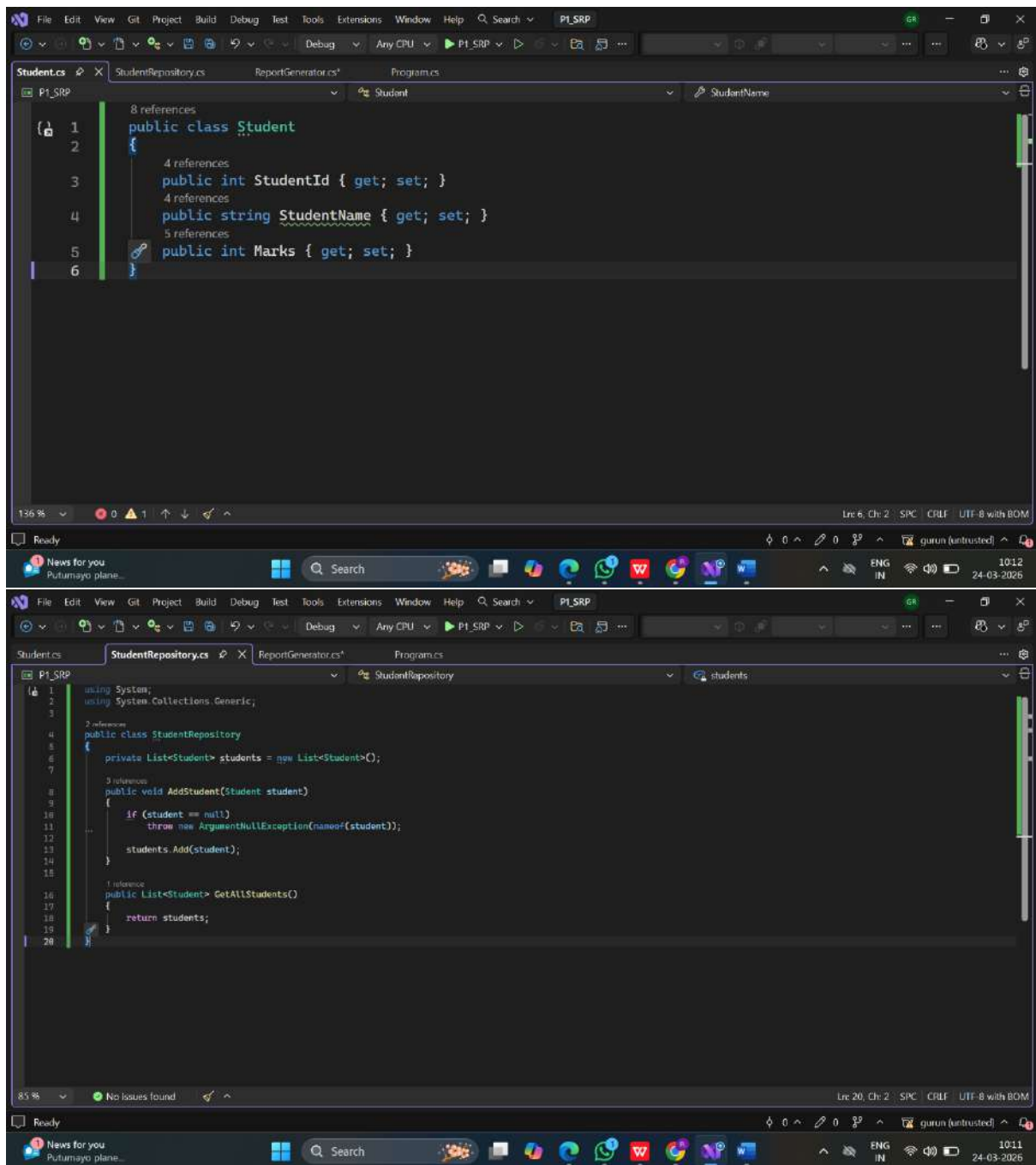
- Use **C# (.NET Console Application)**.
- Each class must have **only one responsibility**.
- Do not mix **data storage and report generation** logic in the same class.

Expectations:

Students should implement at least **three classes**:

- Student
- StudentRepository
- ReportGenerator

Code screenshot:



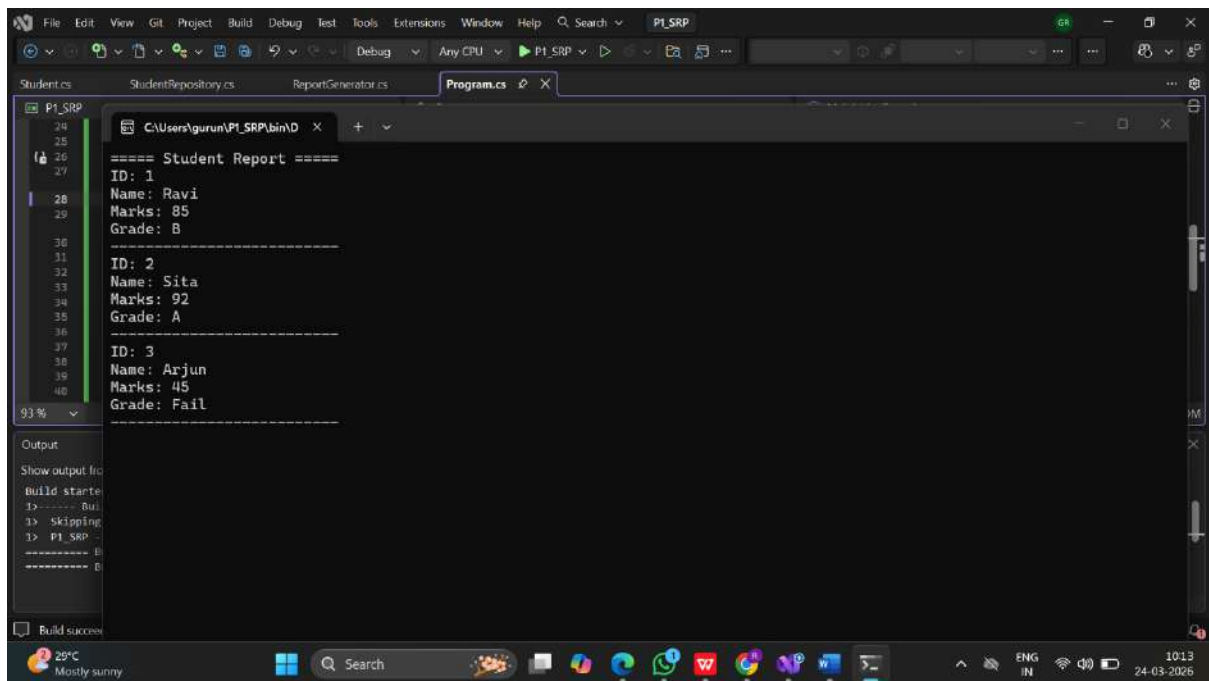
```
File Edit View Git Project Build Debug Test Tools Extensions Window Help P1_SRP
Student.cs StudentRepository.cs ReportGenerator.cs Program.cs
P1_SRP
1 using System;
2 using System.Collections.Generic;
3 public class ReportGenerator
4 {
5     1 reference
6     public void GenerateReport(List<Student> students)
7     {
8         if (students == null || students.Count == 0)
9         {
10             Console.WriteLine("No student data available.");
11             return;
12         }
13         Console.WriteLine("==== Student Report =====");
14         foreach (var student in students)
15         {
16             string grade = GetGrade(student.Marks);
17             Console.WriteLine($"ID: {student.StudentId}");
18             Console.WriteLine($"Name: {student.StudentName}");
19             Console.WriteLine($"Marks: {student.Marks}");
20             Console.WriteLine($"Grade: {grade}");
21         }
22     }
23     1 reference
24     private string GetGrade(int marks)
25     {
26         if (marks >= 90) return "A";
27         if (marks >= 75) return "B";
28         if (marks >= 50) return "C";
29         return "Fail";
30     }
31 }
85 % No issues found Lin: 19, Ch: 50 SPC CRLF UTF-8 with BOM
```



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help P1_SRP
Student.cs StudentRepository.cs ReportGenerator.cs Program.cs
P1_SRP
25 using System;
26
27 0 references
28 class Program
29 {
30     0 references
31     static void Main(string[] args)
32     {
33         StudentRepository repo = new StudentRepository();
34         ReportGenerator reportGenerator = new ReportGenerator();
35
36         // Adding students
37         repo.AddStudent(new Student { StudentId = 1, StudentName = "Ravi", Marks = 85 });
38         repo.AddStudent(new Student { StudentId = 2, StudentName = "Sita", Marks = 92 });
39         repo.AddStudent(new Student { StudentId = 3, StudentName = "Arjun", Marks = 45 });
40
41         // Generate report
42         var students = repo.GetAllStudents();
43         reportGenerator.GenerateReport(students);
44
45         Console.ReadLine();
46     }
47 }
93 % No issues found Lin: 24, Ch: 17 SPC CRLF UTF-8 with BOM
```



Output:



```
==== Student Report ====
ID: 1
Name: Ravi
Marks: 85
Grade: B
-----
ID: 2
Name: Sita
Marks: 92
Grade: A
-----
ID: 3
Name: Arjun
Marks: 45
Grade: Fail
-----
```

Problem: 2- OCP – Open Closed Principle

Scenario: Discount Calculation System

An e-commerce system calculates discounts for different customer types:

- Regular Customer
- Premium Customer
- VIP Customer

The system should allow **adding new discount types without modifying existing code.**

Requirements:

1. Create an abstract class or interface:

IDiscountStrategy

2. Implement discount classes:

- RegularCustomerDiscount
- PremiumCustomerDiscount
- VipCustomerDiscount

3. Each class should implement a method:

CalculateDiscount(double amount)

Technical Constraints:

- Use interface or abstract class
- Existing classes should not be modified when adding new discounts
- Follow Open for Extension, Closed for Modification

Expectations:

Students should implement:

- IDiscountStrategy
- 3 Discount Classes
- A class that calculates the final price

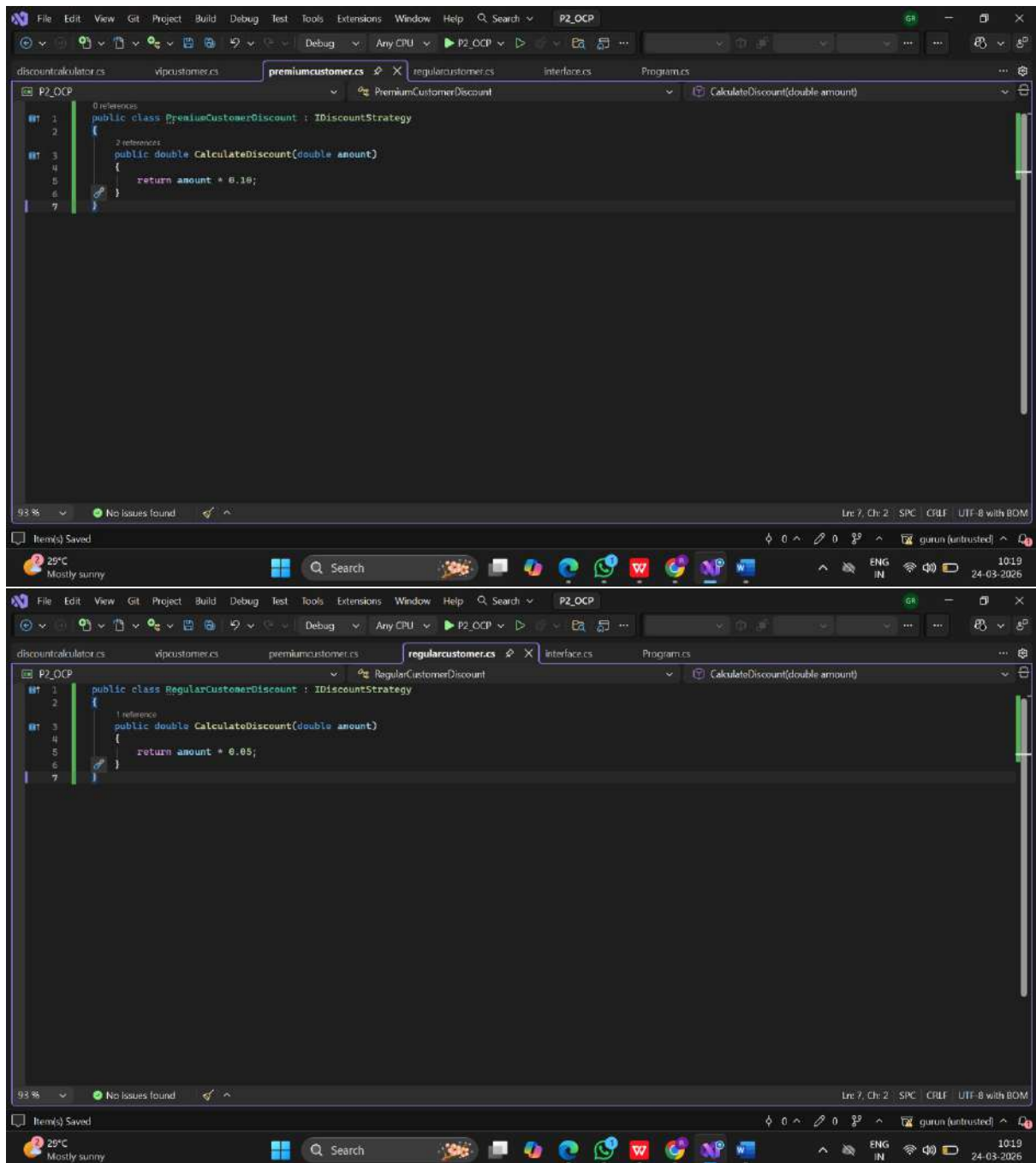
Code Screenshot:

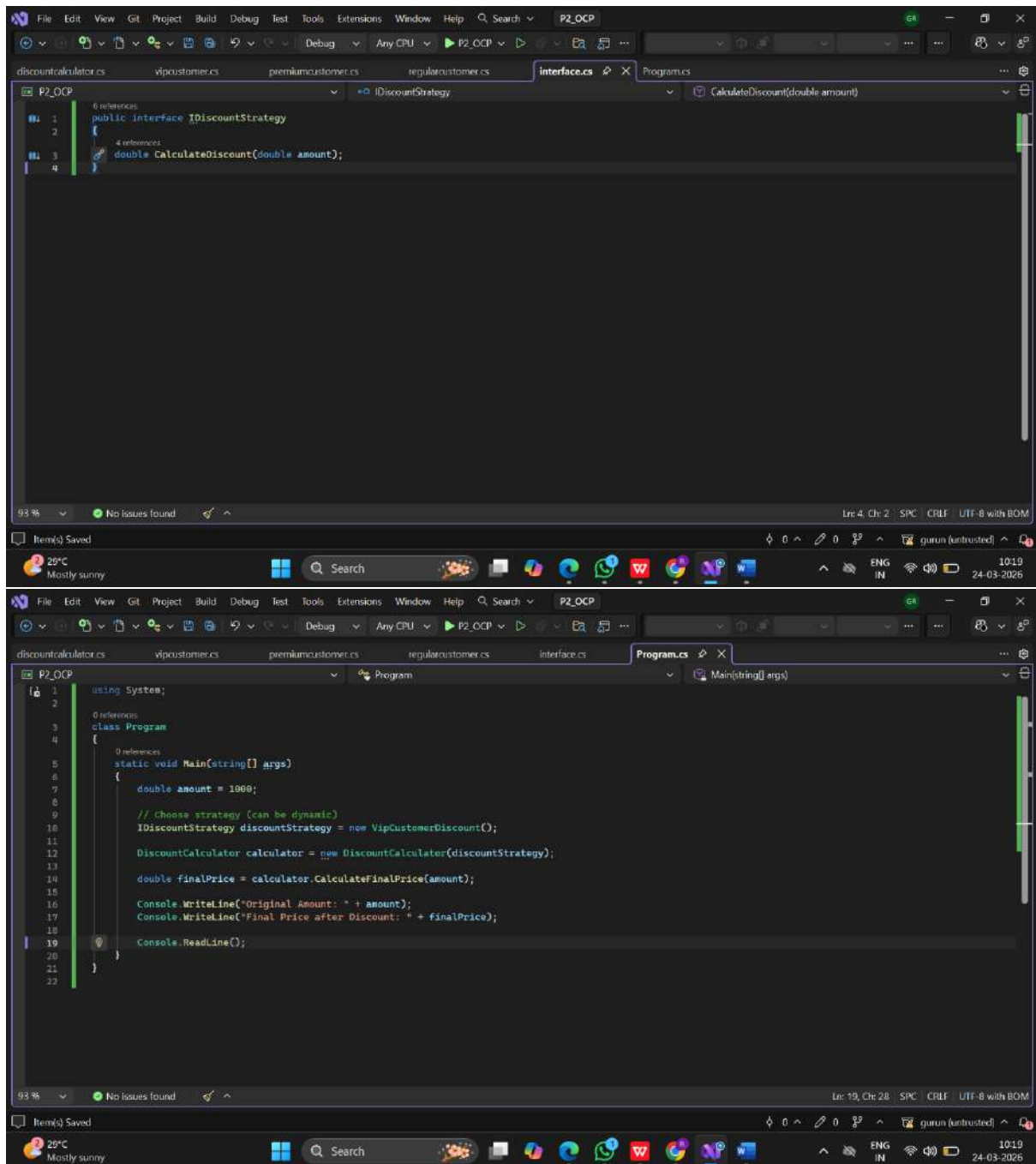
```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P2_OCP
discountcalculator.cs vipcustomer.cs premiumcustomer.cs regularcustomer.cs interface.cs Program.cs
P2_OCP
using System;
public class DiscountCalculator
{
    private readonly IDiscountStrategy _discountStrategy;
    public DiscountCalculator(IDiscountStrategy discountStrategy)
    {
        _discountStrategy = discountStrategy;
        ?? throw new ArgumentNullException(nameof(discountStrategy));
    }
    public double CalculateFinalPrice(double amount)
    {
        if (amount < 0)
            throw new ArgumentException("Amount cannot be negative");
        double discount = _discountStrategy.CalculateDiscount(amount);
        return amount - discount;
    }
}
```

93 % No issues found Lrc 21, Ch: 2 SPC CRLF UTF-8 with BOM

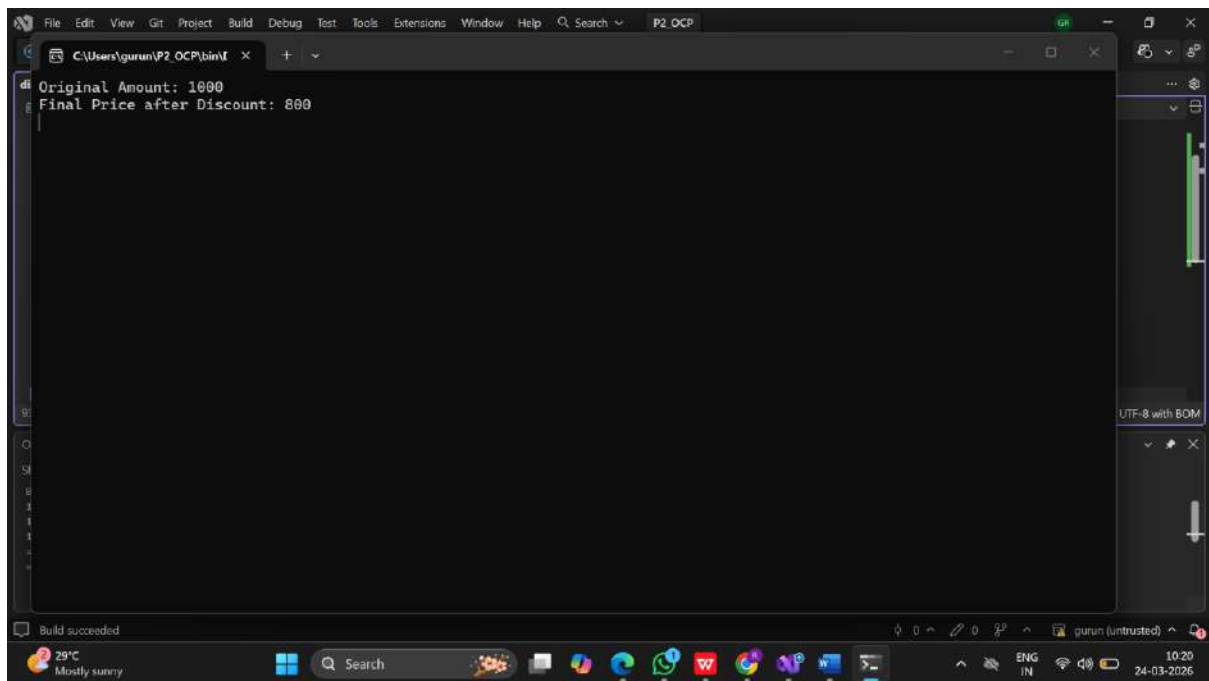
```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P2_OCP
discountcalculator.cs vipcustomer.cs premiumcustomer.cs regularcustomer.cs interface.cs Program.cs
P2_OCP
0 references
public class VipCustomerDiscount : IDiscountStrategy
{
    2 references
    public double CalculateDiscount(double amount)
    {
        return amount * 0.20;
    }
}
```

93 % No issues found Lrc 7, Ch: 2 SPC CRLF UTF-8 with BOM





Output:



```
Original Amount: 1000
Final Price after Discount: 800
```

Problem 3- LSP – Liskov Substitution Principle

Scenario: Shape Area Calculator

A graphics application calculates the **area of different shapes**.

Any derived class should be able to **replace the base class without breaking functionality**.

Requirements:

1. Create a base class or interface:
 - Shape
2. Derived classes:
 - Rectangle
 - Circle
3. Each shape should implement:
 - CalculateArea()
4. A method should accept **Shape object** and calculate area.

Technical Constraints:

- Use method overriding
- Derived classes must not break base class behavior

Expectations:

Students should demonstrate that the program works correctly when:

- Rectangle object is used
- Circle object is used

Code Screenshot:

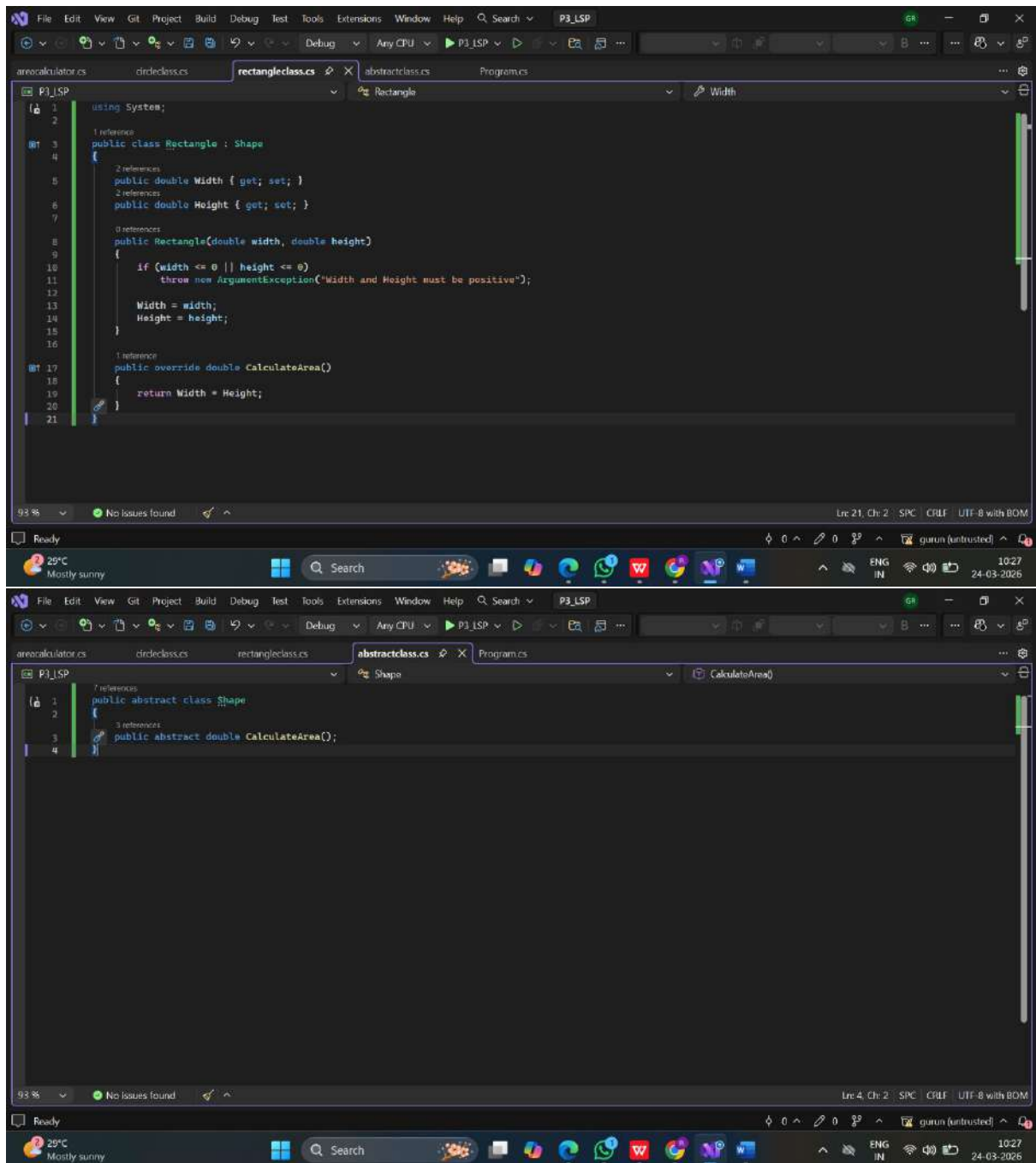
The image displays two screenshots of a Visual Studio IDE, showing C# code for an area calculator application.

Top Screenshot: The code editor shows the `AreaCalculator` class in `areacalculator.cs`. The class has a `PrintArea` method that takes a `Shape` object as a parameter. The method checks if the shape is null and throws an `ArgumentNullException` if so. Otherwise, it calls `shape.CalculateArea()` and prints the result to the console.

```
1 using System;
2
3 2 references
4 public class AreaCalculator
5 {
6     2 references
7     public void PrintArea(Shape shape)
8     {
9         if (shape == null)
10             throw new ArgumentNullException(nameof(shape));
11         Console.WriteLine("Area: " + shape.CalculateArea());
12     }
13 }
```

Bottom Screenshot: The code editor shows the `Circle` class in `circleclass.cs`. The class inherits from `Shape` and implements the `CalculateArea` method. It also has a `Radius` property.

```
1 using System;
2
3 1 reference
4 public class Circle : Shape
5 {
6     3 references
7     public double Radius { get; set; }
8
9     0 references
10    public Circle(double radius)
11    {
12        if (radius <= 0)
13            throw new ArgumentException("Radius must be positive");
14        Radius = radius;
15    }
16
17    1 reference
18    public override double CalculateArea()
19    {
20        return Math.PI * Radius * Radius;
21    }
22 }
```



```
1 using System;
2
3 namespace P3_LSP
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             AreaCalculator calculator = new AreaCalculator();
10
11             Shape rectangle = new Rectangle(10, 5);
12             Shape circle = new Circle(7);
13
14             // LSP in action
15             calculator.PrintArea(rectangle);
16             calculator.PrintArea(circle);
17
18             Console.ReadLine();
19         }
20     }
21 }
```

Output:

```
Area: 50
Area: 153.93804002589985
```

Problem 4- ISP – Interface Segregation Principle

Scenario: Office Printer System

An office has different machines:

- Basic Printer (Print only)
- Advanced Printer (Print + Scan + Fax)

If we create a **single large interface**, basic printers will be forced to implement unnecessary methods.

The task is to **split the interface into smaller interfaces**.

Requirements:

Create the following interfaces:

- IPrinter
- IScanner
- IFax

Classes:

- BasicPrinter → Print only
- AdvancedPrinter → Print + Scan + Fax

Technical Constraints:

- Follow **Interface Segregation Principle**
- Classes should **not implement unnecessary methods**

Expectations:

Students should implement:

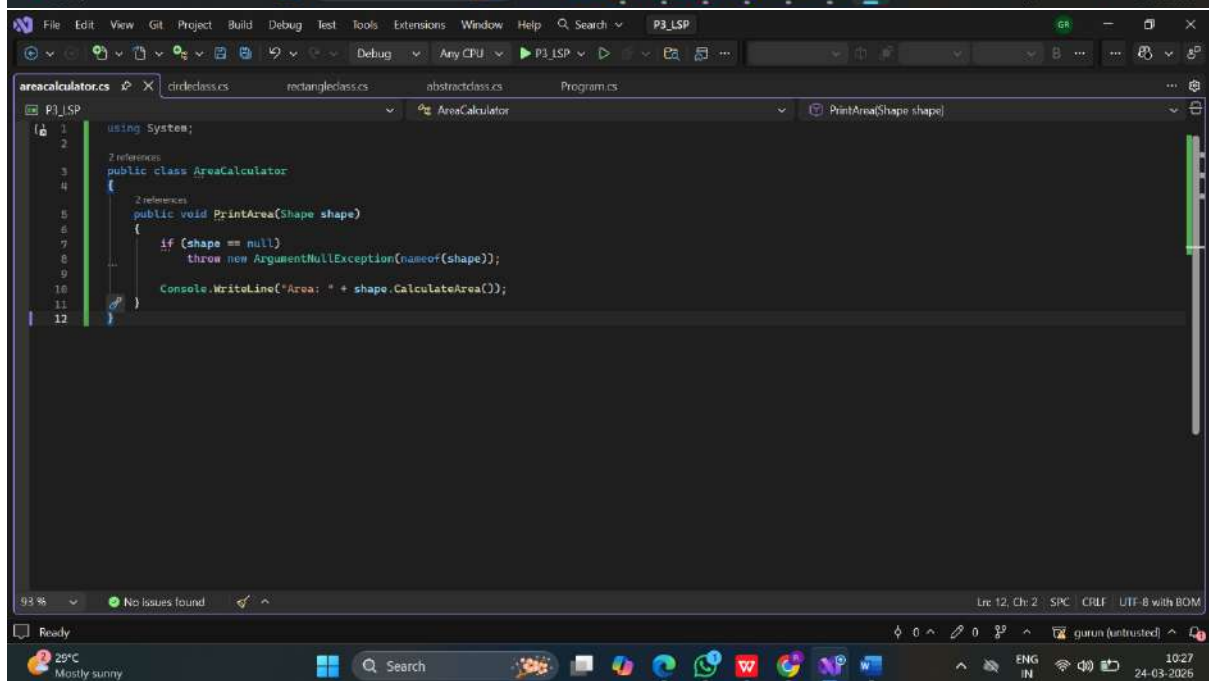
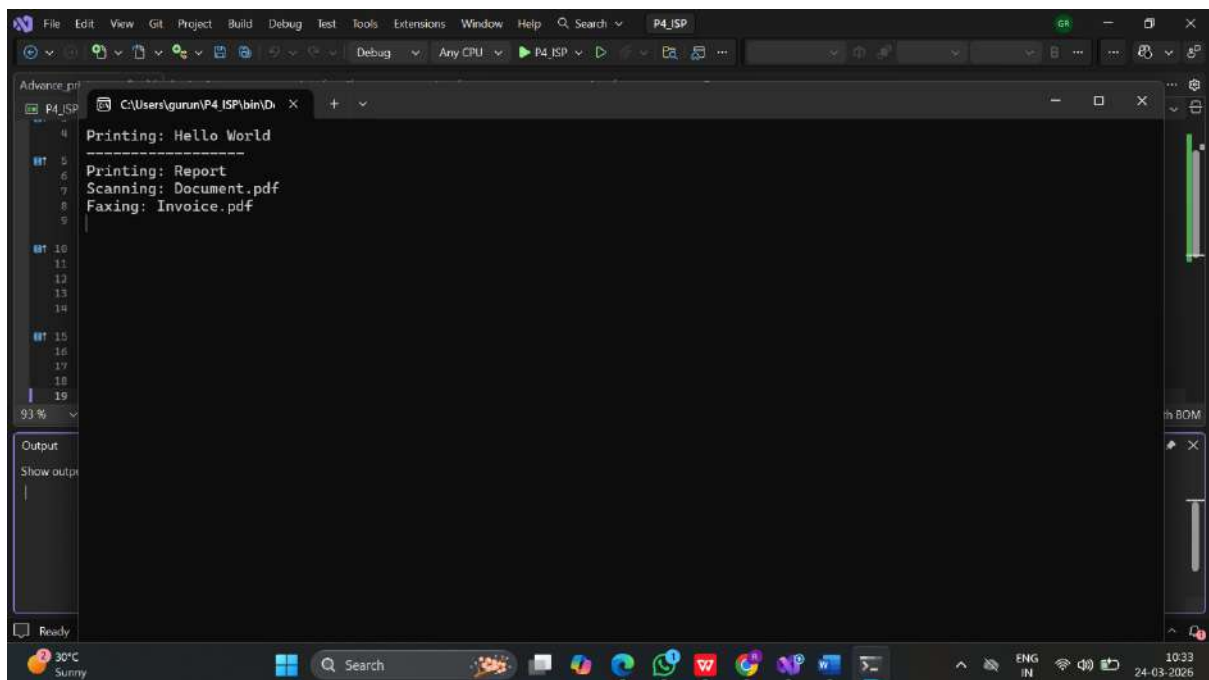
Interfaces

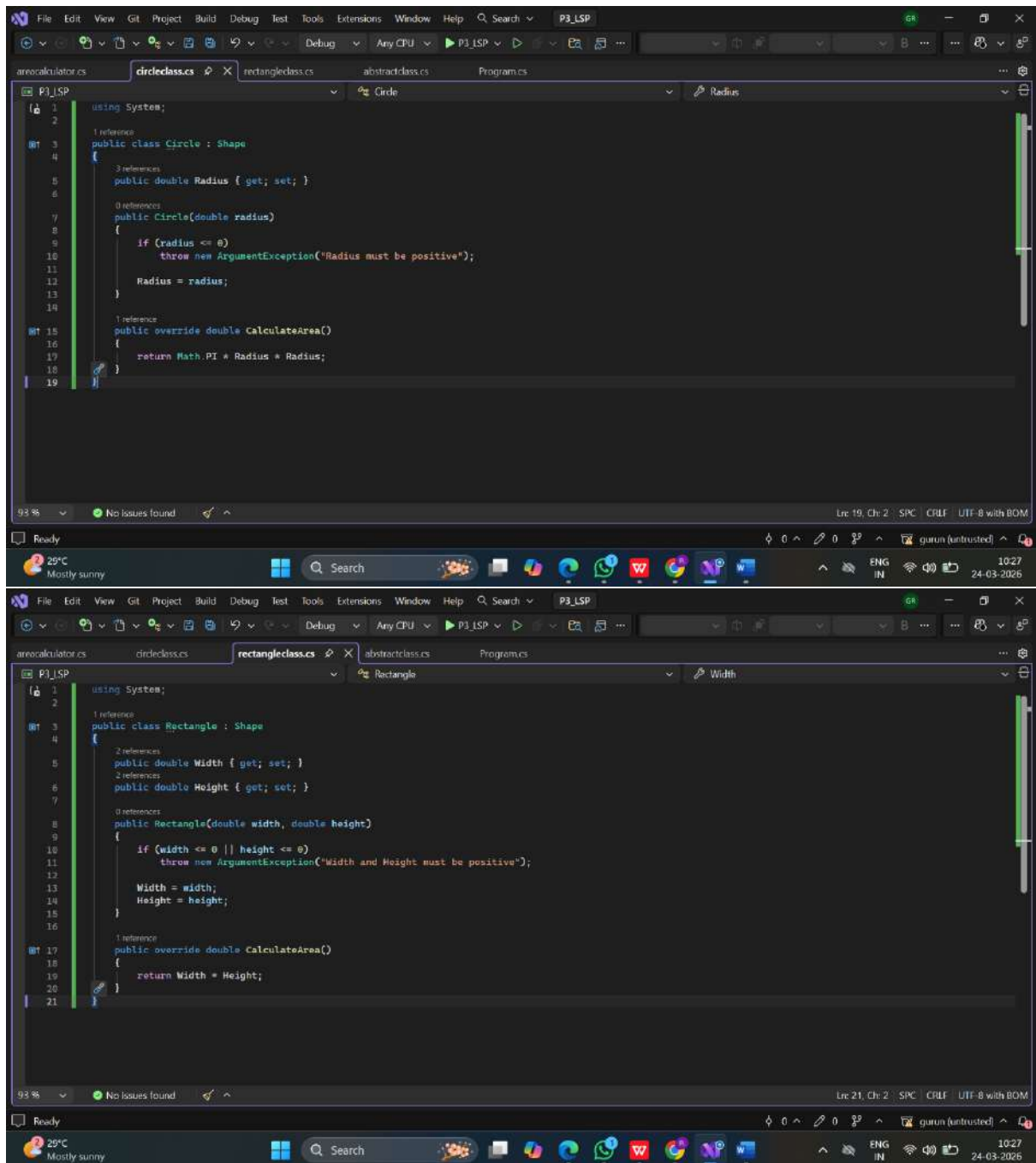
- IPrinter
- IScanner
- IFax

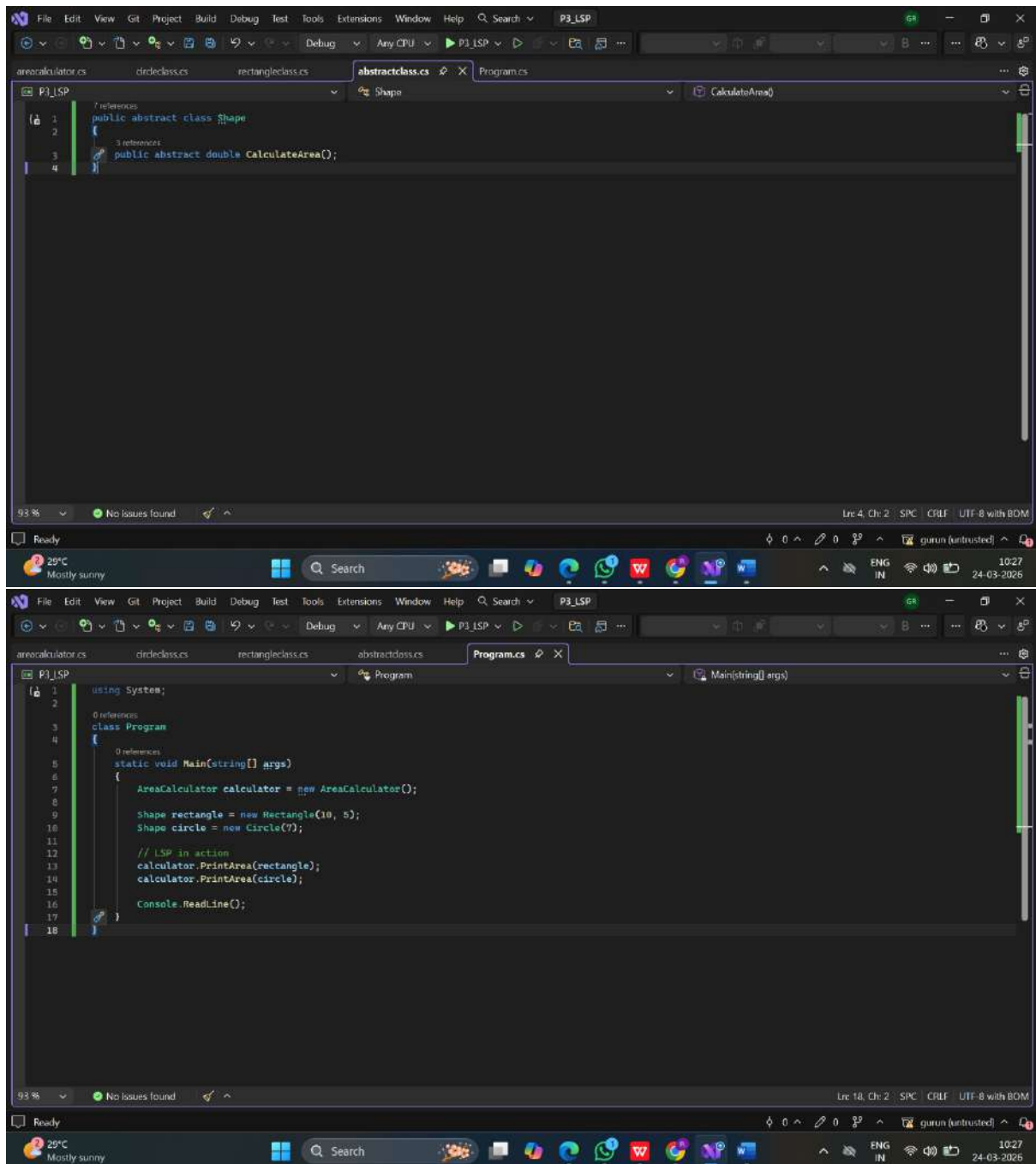
Classes

- BasicPrinter
- AdvancedPrinter

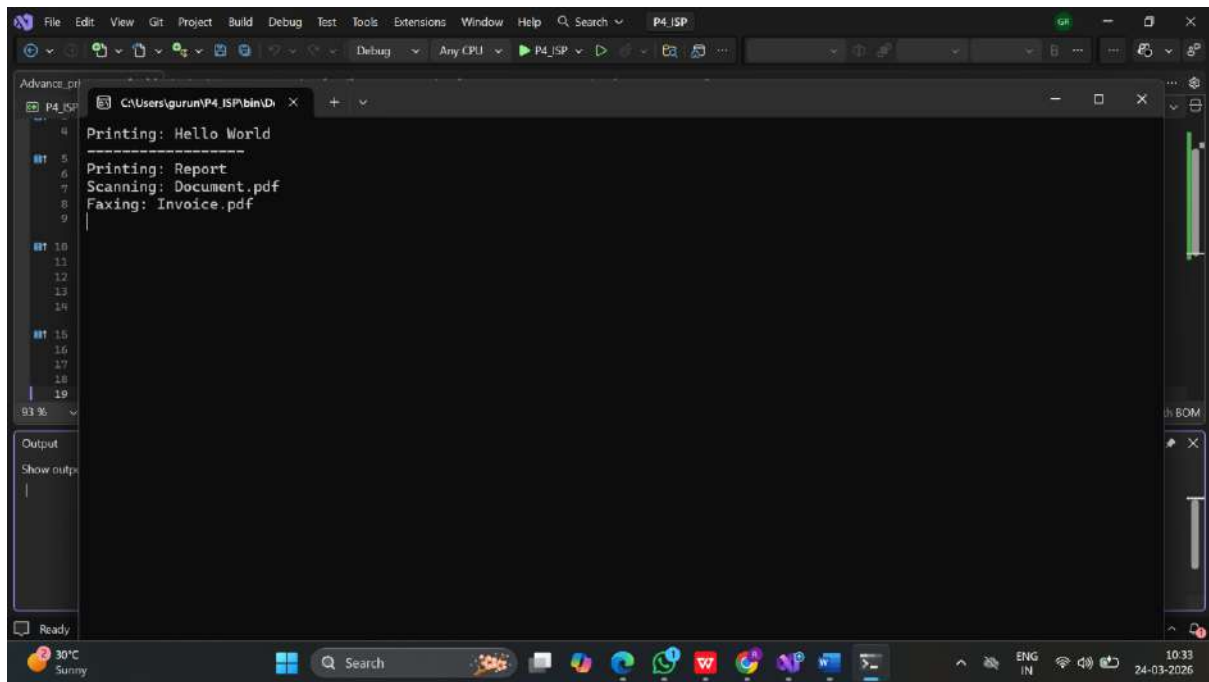
Code Screenshot:







Output:



Problem 5- Implementing Singleton Pattern

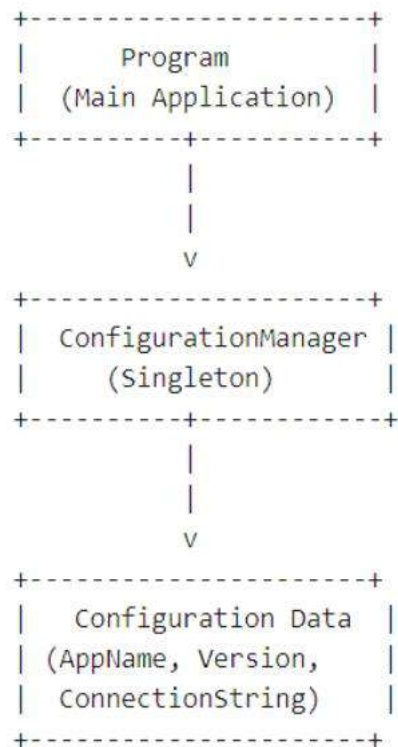
Scenario: Application Configuration Manager

A company is developing a **console-based inventory management system**.

The application needs to read **application settings (e.g., database connection string, app name, version)** from a configuration manager.

Since configuration settings should be **loaded only once and shared across the entire application**, multiple instances of the configuration manager must be prevented.

Therefore, the development team decides to implement the **Singleton Design Pattern**.



Requirements:

Students must implement:

1. A class named **ConfigurationManager**
2. Ensure **only one instance** of the class can be created.
3. Provide a method:

GetInstance()

to retrieve the single object.

4. Store configuration values such as:
 - ApplicationName
 - Version
 - DatabaseConnectionString
5. Demonstrate that multiple calls to GetInstance() return **the same instance**.

Technical Constraints:

- Use **private constructor**.
- Use **static instance variable**.
- Use **thread-safe implementation (basic level optional)**.

- Console application using **C# (.NET)**.

Expectations:

Students should:

- Prevent object creation using new.
- Access instance using:
`ConfigurationManager.GetInstance()`
- Print configuration details from multiple method calls.

Learning Outcome:

After completing this problem, learners will understand:

- Why Singleton is used
- How to restrict object creation
- Global shared objects
- Basic design pattern implementation in C#

Code Screenshots:

configurationmanager.cs x Program.cs

p5_Singleton

```
18 {
19     // Simulate loading config
20     ApplicationName = "Inventory Management System";
21     Version = "1.0.0";
22     DatabaseConnectionString = "Server=localhost;Database=InventoryDB;Trusted_Connection=True;";
23 }
24
25 // Public method to get instance
26 public static ConfigurationManager GetInstance()
27 {
28     // Double-check locking (thread-safe)
29     if (_instance == null)
30     {
31         lock (_lock)
32         {
33             if (_instance == null)
34             {
35                 _instance = new ConfigurationManager();
36             }
37         }
38     }
39     return _instance;
40 }
41 }
```

102 % 0 1 ↑ ↓ ↺ ↻

Line 41, Ch: 2 SPC CRLF UTF-8 with BOM

30°C Sunny

Search

ENG IN 10:37 24-03-2026

configurationmanager.cs x Program.cs

p5_Singleton

```
1 using System;
2
3 public sealed class ConfigurationManager
4 {
5     // Static instance (only one)
6     private static ConfigurationManager _instance;
7
8     // Lock object for thread safety
9     private static readonly object _lock = new object();
10
11     // Properties
12     public string ApplicationName { get; private set; }
13     public string Version { get; private set; }
14     public string DatabaseConnectionString { get; private set; }
15
16     // Private constructor (prevents new)
17     private ConfigurationManager()
18     {
19         // Simulate loading config
20         ApplicationName = "Inventory Management System";
21         Version = "1.0.0";
22         DatabaseConnectionString = "Server=localhost;Database=InventoryDB;Trusted_Connection=True;";
23     }
24 }
```

102 % 0 1 ↑ ↓ ↺ ↻

Line 41, Ch: 2 SPC CRLF UTF-8 with BOM

Ready

30°C Sunny

Search

ENG IN 10:37 24-03-2026

```
1 using System;
2
3 0 references
4 class Program
5 {
6     0 references
7     static void Main(string[] args)
8     {
9         var config1 = ConfigurationManager.GetInstance();
10        var config2 = ConfigurationManager.GetInstance();
11
12        Console.WriteLine("Config 1:");
13        PrintConfig(config1);
14
15        Console.WriteLine("\nConfig 2:");
16        PrintConfig(config2);
17
18        // Check if both instances are same
19        Console.WriteLine("\nAre both instances same? " + (config1 == config2));
20        Console.ReadLine();
21    }
22
23    2 references
24    static void PrintConfig(ConfigurationManager config)
25    {
26        Console.WriteLine("App Name: " + config.ApplicationName);
27        Console.WriteLine("Version: " + config.Version);
28        Console.WriteLine("DB Connection: " + config.DatabaseConnectionString);
29    }
30 }
```

Output:

```
Config 1:
App Name: Inventory Management System
Version: 1.0.0
DB Connection: Server=localhost;Database=InventoryDB;Trusted_Connection=True;

Config 2:
App Name: Inventory Management System
Version: 1.0.0
DB Connection: Server=localhost;Database=InventoryDB;Trusted_Connection=True;

Are both instances same? True
```

Problem 6- Implementing Factory Pattern

Scenario: Notification Service

A company application sends **notifications to users** using different communication channels:

- Email
- SMS

- Push Notification

The application should **not directly create objects using new**.

Instead, a **Factory class** should determine which notification service to create.

Requirements:

Students must implement:

- **Interface**

INotification

- **Method:**

Send(string message)

Classes

- EmailNotification
- SMSNotification
- PushNotification

Each class should implement **INotification**.

Factory Class

NotificationFactory

Method:

CreateNotification(string type)

Example:

CreateNotification("email")

CreateNotification("sms")

CreateNotification("push")

Technical Constraints

- Use **interface-based design**.
- Client should **not instantiate concrete classes directly**.
- Use **Factory Pattern** to create objects.
- Language: **C# Console Application**.

Expectations:

Students should demonstrate:

```
NotificationFactory factory = new NotificationFactory();
```

```
var notification = factory.CreateNotification("email");
```

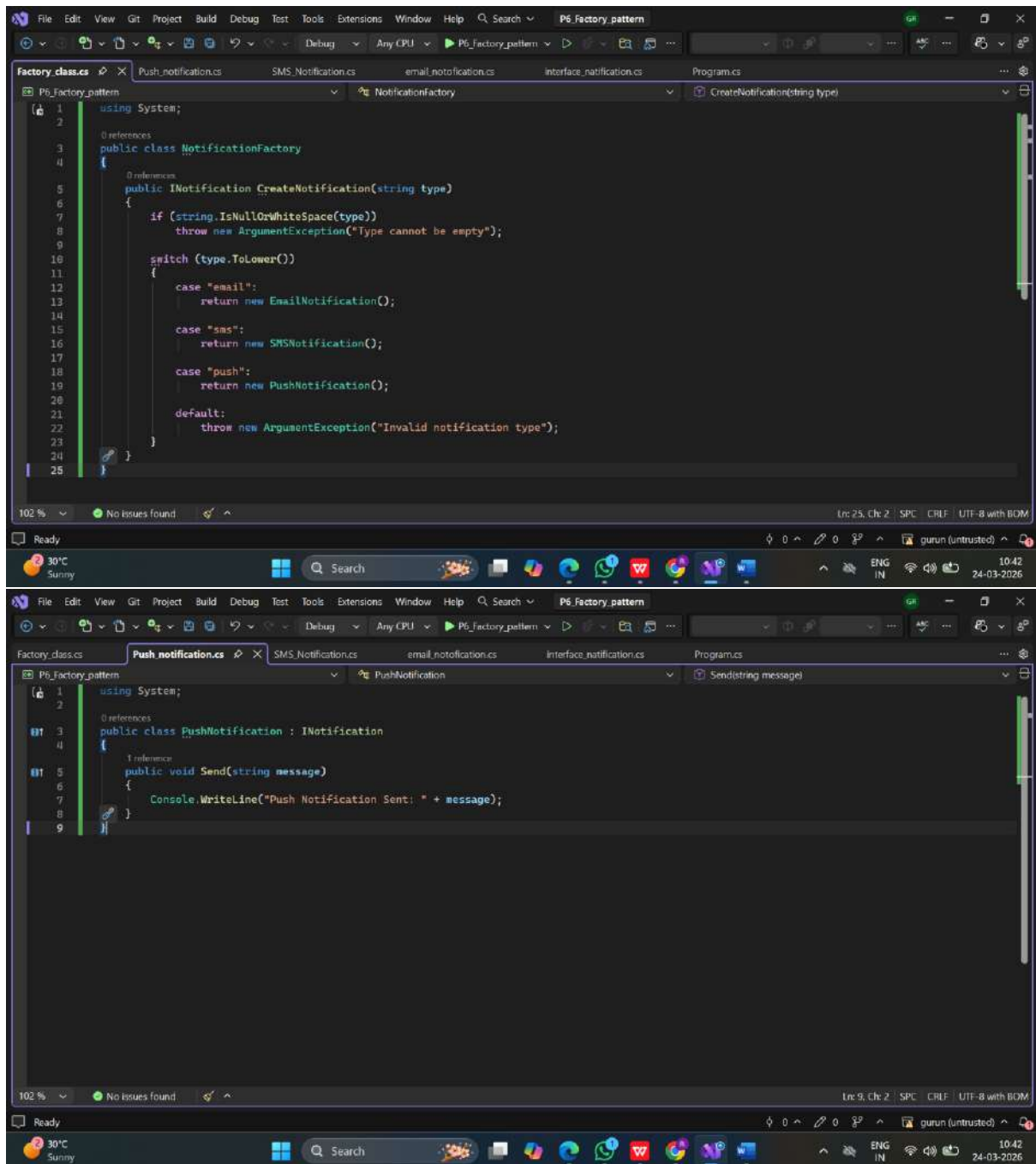
```
notification.Send("Welcome to our service!");
```

Learning Outcome:

Students will learn:

- Decoupling object creation from usage
- Interface-based programming
- Open/Closed Principle basics
- Real-world object creation management

Code Screenshot:



File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P6_Factory_pattern

Factory_classes Push_notifications SMS_Notification.cs email_notification.cs interface_notification.cs Program.cs

```
1 using System;
2
3 public class SMSNotification : INotification
4 {
5     public void Send(string message)
6     {
7         Console.WriteLine("SMS Sent: " + message);
8     }
9 }
```

102 % No issues found. Lnc 9, Ch: 2 SPC CRLF UTF-8 with BOM

Ready 30°C Sunny Search ENG IN 10:42 24-03-2026

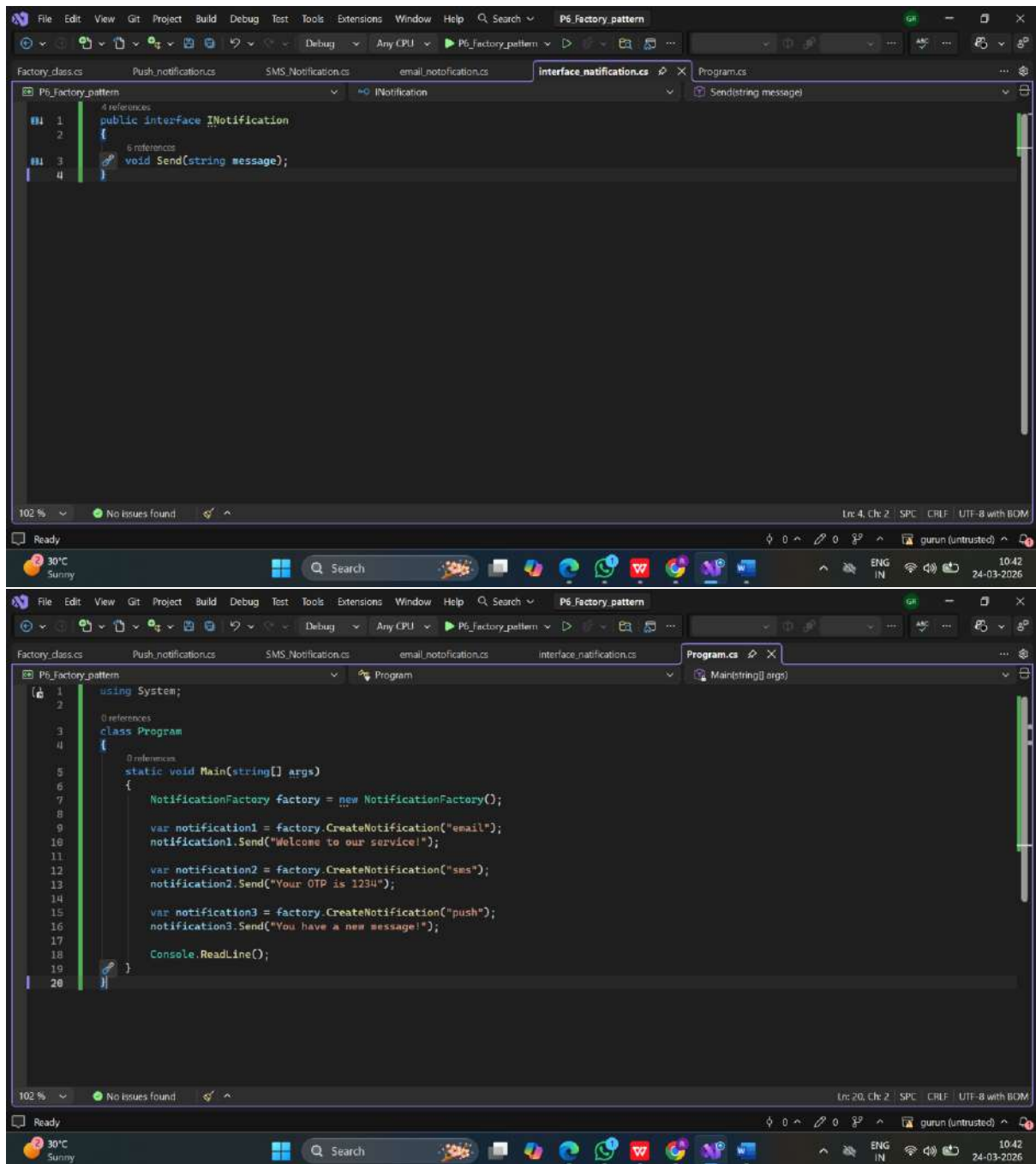
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P6_Factory_pattern

Factory_classes Push_notifications SMS_Notification.cs email_notification.cs interface_notification.cs Program.cs

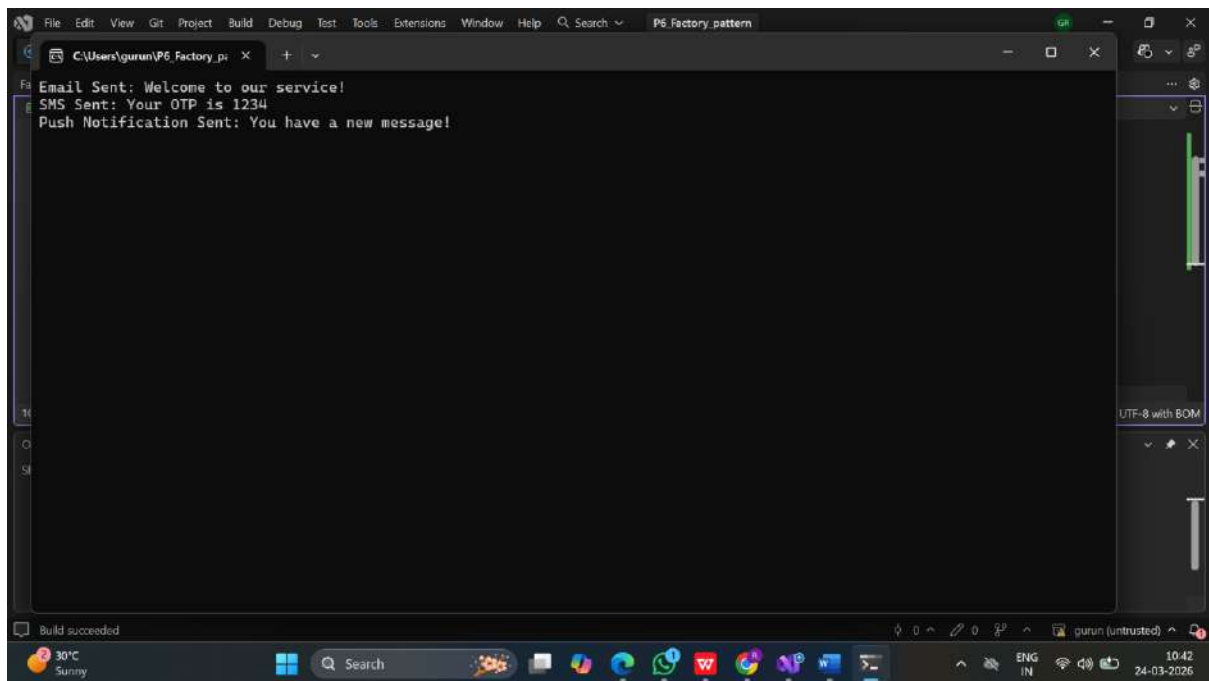
```
1 using System;
2
3 public class EmailNotification : INotification
4 {
5     public void Send(string message)
6     {
7         Console.WriteLine("Email Sent: " + message);
8     }
9 }
```

102 % No issues found. Lnc 9, Ch: 2 SPC CRLF UTF-8 with BOM

Ready 30°C Sunny Search ENG IN 10:42 24-03-2026



Output:



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P6 Factory.pattern
C:\Users\gurun\P6_Factory.pr
Email Sent: Welcome to our service!
SMS Sent: Your OTP is 1234
Push Notification Sent: You have a new message!
Build succeeded
30°C Sunny
Search
gurun (untrusted)
10:42
24-03-2026
```

Problem 7- Implementing Repository Pattern

Scenario: Student Data Management System

A training institute needs a system to manage **student information**.

Instead of directly writing database access code in the main program, the development team decides to use the **Repository Pattern**.

The repository will act as a **data access layer** between the application and the data source.

For simplicity, student data will be stored in a **List collection**.

Requirements:

Create the following components:

Entity Class:

Student

Properties:

StudentId

StudentName

Course

Repository Interface:

IStudentRepository

Methods:

AddStudent(Student student)
GetAllStudents()
GetStudentById(int id)
DeleteStudent(int id)

Repository Implementation:

StudentRepository

Store data using:

List<Student>

Main Program

Demonstrate:

- Adding students
- Viewing students
- Finding student by ID
- Deleting student

Expectations:

Students should implement separation between:

Business Logic

↓

Repository Layer

↓

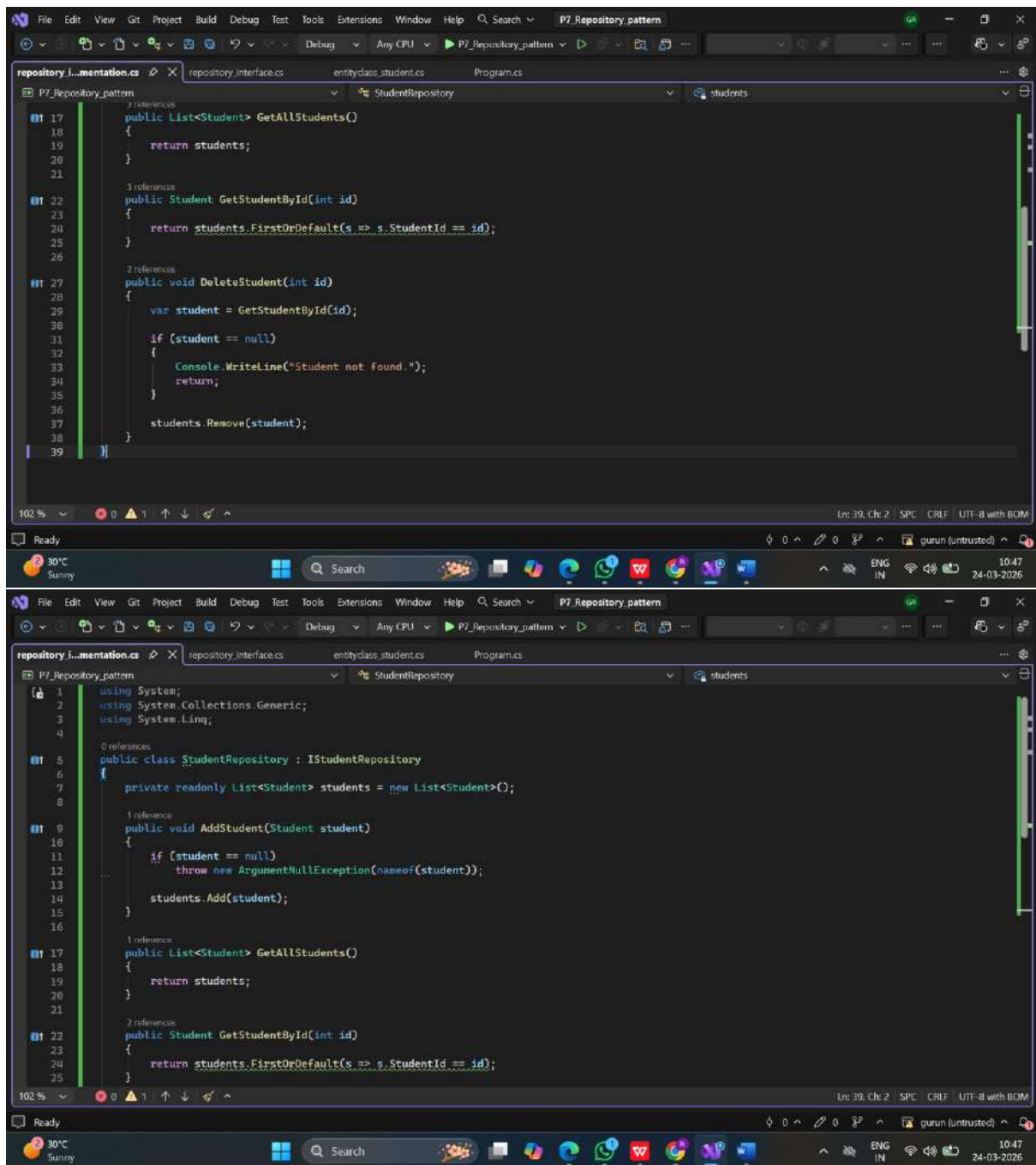
Data Storage

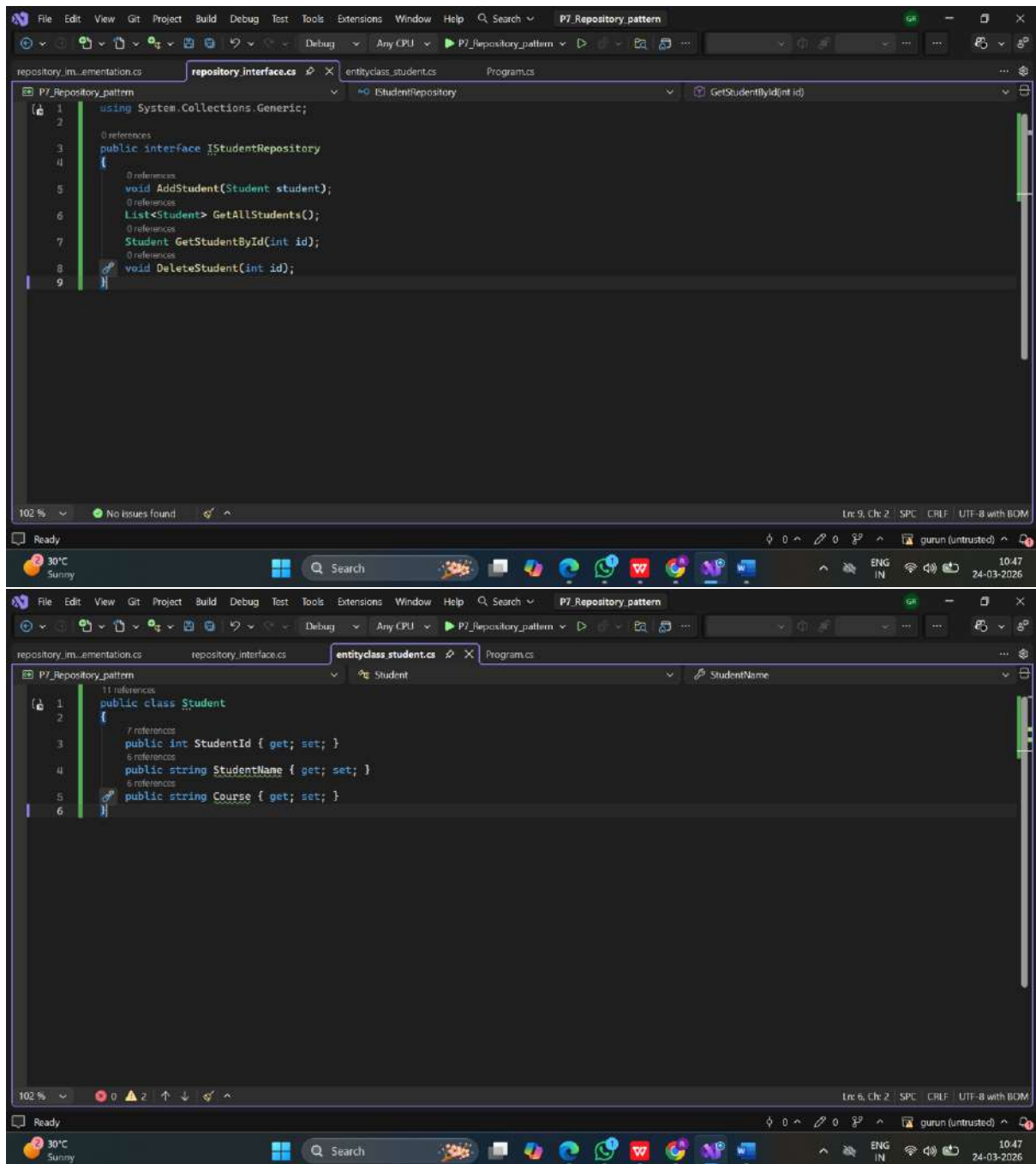
Learning Outcome:

Students will understand:

- Separation of concerns
- Data access abstraction
- Clean architecture basics
- How repositories simplify data operations

Code Screenshots:





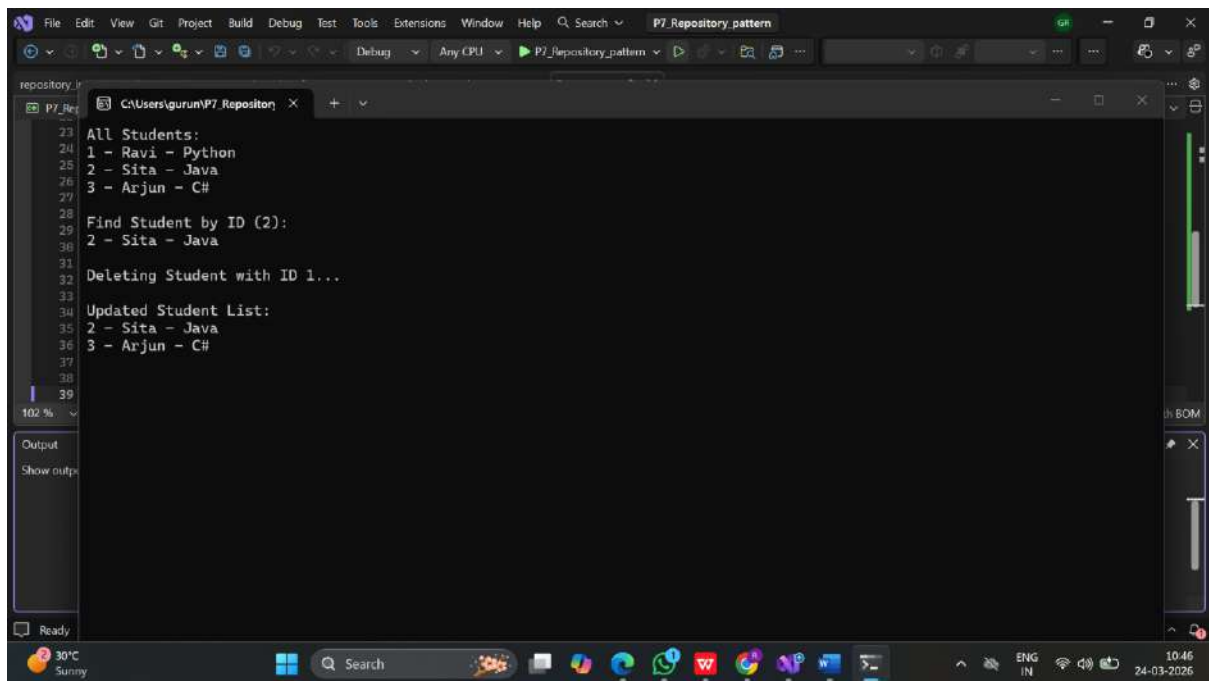
```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P7_Repository_pattern
Debug Any CPU P7_Repository_pattern
repository_im_ementation.cs repository_interface.cs entityclass_student.cs Program.cs
P7_Repository_pattern
17 {
18     Console.WriteLine($"{student.StudentId} - {student.StudentName} - {student.Course}");
19 }
20
21 Console.WriteLine("\nFind Student by ID (2):");
22 var found = repository.GetStudentById(2);
23 if (found != null)
24     Console.WriteLine($"{found.StudentId} - {found.StudentName} - {found.Course}");
25
26 // Delete Student
27 Console.WriteLine("\nDeleting Student with ID 1...");
28 repository.DeleteStudent(1);
29
30 // View Again
31 Console.WriteLine("\nUpdated Student List:");
32 foreach (var student in repository.GetAllStudents())
33 {
34     Console.WriteLine($"{student.StudentId} - {student.StudentName} - {student.Course}");
35 }
36
37 Console.ReadLine();
38 }
39 }
```



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P7_Repository_pattern
Debug Any CPU P7_Repository_pattern
repository_im_ementation.cs repository_interface.cs entityclass_student.cs Program.cs
P7_Repository_pattern
1 using System;
2
3 // references
4 class Program
5 {
6     // references
7     static void Main(string[] args)
8     {
9         IStudentRepository repository = new StudentRepository();
10
11         // Add Students
12         repository.AddStudent(new Student { StudentId = 1, StudentName = "Ravi", Course = "Python" });
13         repository.AddStudent(new Student { StudentId = 2, StudentName = "Sita", Course = "Java" });
14         repository.AddStudent(new Student { StudentId = 3, StudentName = "Arjun", Course = "C#" });
15
16         // View All Students
17         Console.WriteLine("All Students:");
18         foreach (var student in repository.GetAllStudents())
19         {
20             Console.WriteLine($"{student.StudentId} - {student.StudentName} - {student.Course}");
21         }
22
23         Console.WriteLine("\nFind Student by ID (2):");
24         var found = repository.GetStudentById(2);
25         if (found != null)
26             Console.WriteLine($"{found.StudentId} - {found.StudentName} - {found.Course}");
27
28         // Delete Student
29         Console.WriteLine("\nDeleting Student with ID 1...");
30     }
31 }
```



Output:



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P7.Repository.pattern
Debug Any CPU P7.Repository.pattern
repository.p C:\Users\gurun\P7.Repository
23 All Students:
24 1 - Ravi - Python
25 2 - Sita - Java
26 3 - Arjun - C#
27
28 Find Student by ID (2):
29 2 - Sita - Java
30
31 Deleting Student with ID 1...
32
33 Updated Student List:
34 2 - Sita - Java
35 3 - Arjun - C#
36
37
38
39
102 %
Output
Show output
Ready
30°C Sunny
Search
ENG IN
10:46 24-03-2026
```